

Nomadic Online TSP On the Circle

Jatinder Sahota

September 5, 2025

Abstract

The Online TSP is a variation of the Traveling Salesman Problem where the salesman (server) receives information overtime rather than at the start of execution. We consider the problem on the Ring (circle) metric space, where the server can only move along the perimeter of a circle. In addition, the server is not required to return to the origin upon finishing any tours. Our goal is to close the gap between the best-known algorithm for the problem and a proven lower bound. We provide a conjectured optimal solution to the problem.

1 Preliminaries

The **homing** variant of this problem requires that the server return to the origin after serving M -many requests, where M is the total number of requests submitted. In practice, the server would have to return to the origin after any tour. The **nomadic** variant relaxes this constraint. Difficulty in the problem stems from a lack of clairvoyance – the server does not know which requests will be arriving next, nor at what time. Hence, the server must make decisions without future knowledge. This problem was coined by Ausiello [1].

Points lie on a circle of circumference C , and distances between points are considered to be the shortest arc between them.

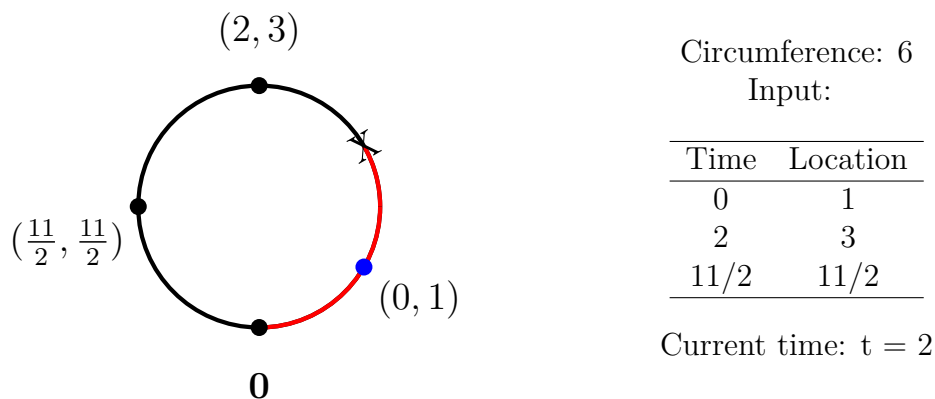


Figure 1: An example; requests are labeled as (time, location) pairs, and the server is the cross.

Definition 1.1. A request sequence (σ) of size $M \in \mathbb{N}$ is defined to be:

$$\sigma := \{\sigma_i = (t_i, a_i) : 0 \leq a_i < C, t_i \geq 0\}_{i=1}^M$$

where C is the circumference of the ring.

To serve a request σ_i , the server must visit a_i at time $t \geq t_i$. We define the "cost" of our algorithm to be the total time spent to serve M -many requests, rather than total travel distance.

Definition 1.2. For all request sequences σ , the **competitive ratio** (ρ) of an online algorithm ALG is defined as:

$$\rho := \sup_{\sigma} \frac{Z^{ALG}}{Z^{OPT}}$$

where OPT is the optimal algorithm, and Z^{ALG} and Z^{OPT} are the costs of ALG and OPT on input σ , respectively.

Definition 1.3. Let $d(x, y) :=$ the length of the shorter arc connecting points x and y .

2 Previous Work

The following works showcase the state of the problem:

- **Jawgal et al. [3]** prove a lower bound of $\rho \geq 2$ on the competitive ratio.
- **Jawgal et al. [3]** find an algorithm with competitive ratio $\rho = 2.5$.
- **Blom et al. [2]** show that in general, waiting is better than being zealous (not waiting).

Citations for the papers are in the references.

3 Results

The current best known algorithm does not wait by construction. A natural direction for improvement is to let the server idle while there are requests to serve.

To ensure the algorithm makes the best decision, the server waits as long as possible before serving the most recent released request, and follows a locally-optimal tour.

Smartly Wait And Go is called when a request outside the current tour is released.

Input: All currently-unserved requests $\sigma(t)$, the current time t , the current position of the online server $p(t)$.

Output: The tour for the server to walk (T^{OL}).

- 1: $I(t) :=$ shortest interval covering all unserved requests at time t .
 - 2: $T^G(t) :=$ tour that maximally waits from time 0 and optimally serves $I(t)$ from origin.
 - 3: $Z^G(t) :=$ cost of $T^G(t)$.
 - 4: $t_0 := 2Z^G(t) - d(p(t), e(t)) - \ell(t)$ $\triangleright$ Latest time to follow T^G so that $\rho = 2$
 - 5: **If** $t_0 \geq t$ (i.e. time t_0 has not passed) **then:**
 - 6: $T^{OL} :=$ wait until t_0 then walk $I(t)$ through $e(t)$.
 - 7: **Else**
 - 8: $T^{OL} :=$ minimum-cost tour of: {serve $I(t)$ through $\sigma^L(t)$, serve $I(t)$ through $\sigma^R(t)$ }
 - 9: **return** T^{OL}
-

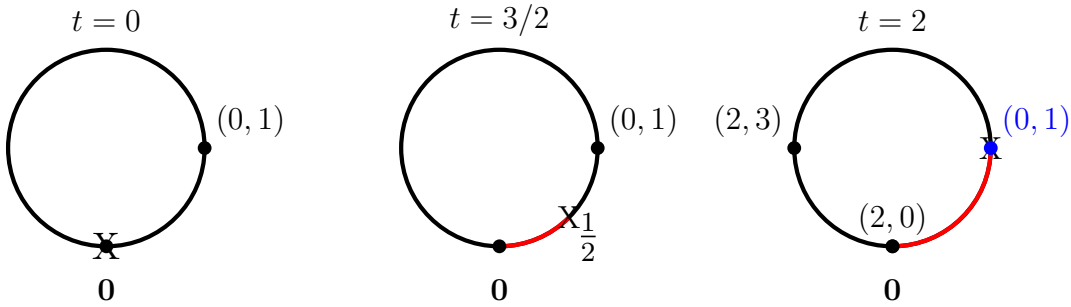


Figure 2: An example demonstrating the algorithm. The server begins moving at $t = 1$.

Definition 3.1. Let $I(t) = [\sigma^L(t), \sigma^R(t)]$ be the smallest interval (arc) covering all unserved requests at time t with endpoints $\sigma^L(t) = (t_L, a_L)$ and $\sigma^R(t) = (t_R, a_R)$. Denote the length of the arc as $\ell(t)$, and the most recent-released request $\sigma_m = (t_m, a_m)$.

Definition 3.2. Let $\mathcal{T}(t)$ be the set of tours starting at the origin that serve $I(t)$. Let $w(T)$ denote the amount of time any tour T idles at the origin starting from time 0. Define $T^G(t)$ as:

$$T^G(t) := \arg \max_{T \in \mathcal{T}(t)} \{w(T) : T \text{ serves } I(t) \text{ optimally}\}$$

The first endpoint of $I(t)$ that $T^G(t)$ visits will be denoted $e(t)$.

Definition 3.3. Let t_0 denote the time in execution such that:

$$t_0 + d(p(t), e(t)) + \ell(t) = 2 \times Z^G(t)$$

where $Z^G(t)$ is the cost of $T^G(t)$ and $p(t)$ is the position of the server at time t .

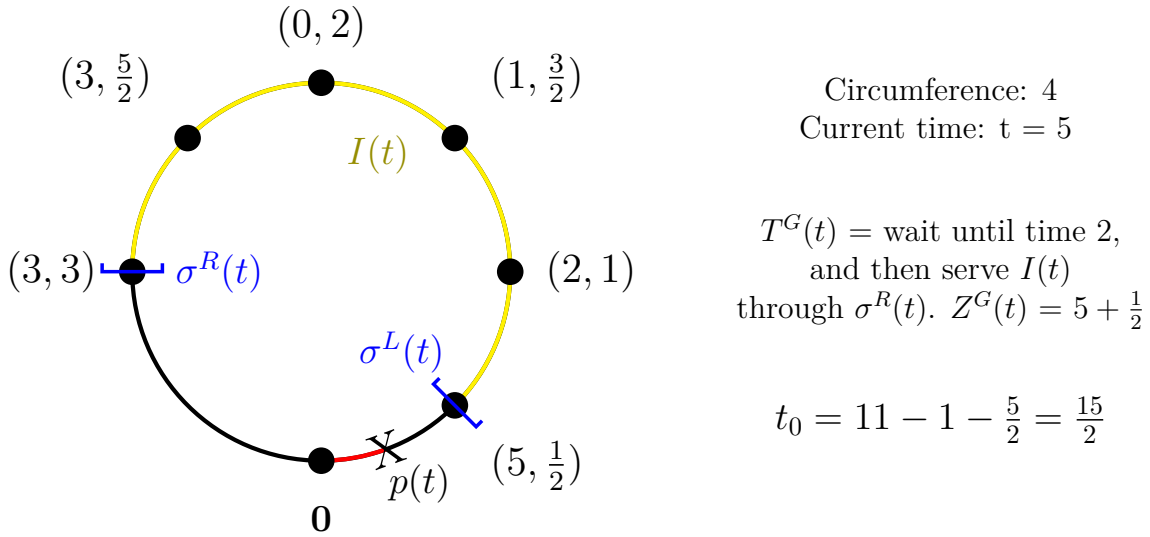


Figure 3: An example illustrating the definitions given before. The yellow arc represents the interval $I(t)$, and the endpoints are labeled. $p(t)$ denotes the position of the server at time $t = 5$, also marked by an X.

Lemma 3.4. For all t and any request sequence, $Z^G(t) \leq Z^{OPT}(t)$, where $T^{OPT}(t)$ optimally serves all requests released up to time t .

Proof sketch: Follows from the definition of $T^G(t)$ and the fact that $T^{OPT}(t)$ must serve at least all of $I(t)$. $\square$

Lemma 3.5. *Let $c(t) = \min\{d(p(t), \sigma^R(t)), d(p(t), \sigma^L(t))\}$. If $t > t_0$ and $L(t) := (t - t_0) + c(t) - d(0, e(t)) > 0$, then,*

$$Z^{ALG}(t) = 2 \times Z^G(t) + L(t)$$

Proof sketch:

- $t - t_0$ is how much time has passed since time t_0 .
- $c(t) - d(0, e(t))$ is the difference in distance that our server travels to the closer endpoint compared to $T^G(t)$ and $e(t)$. $\square$

Proposition 3.6. *Smartly Wait And Go is 2-competitive.*

Intuition: If $t \leq t_0$, then we can afford to idle. Hence, $p(t_0) = p(t)$ and $I(t_0) = I(t)$. Applying the definition of t_0 and Lemma 1:

$$Z^{ALG}(t) = t_0 + d(p(t), e(t)) + \ell(t) = 2 \times Z^G(t) \leq 2 \times Z^{OPT}(t)$$

Otherwise, $t > t_0$, we need only consider the instance constructed in Lemma 2, since otherwise we avoid being late by serving $I(t)$ through the nearer endpoint.

Intuitively, for the server to be placed in such a situation, requests must have been placed prior that moved the server to their current position. We can offset these extra costs since OPT will serve those requests too.

References

- [1] Giorgio Ausiello, Esteban Feuerstein, Stefano Leonardi, Leen Stougie, and Maurizio Talamo. *Algorithms for the On-line Travelling Salesman*. 08 1999.
- [2] Michiel Blom, Sven O. Krumke, Willem de Paepe, and Leen Stougie. The online-tsp against fair adversaries. In Giancarlo Bongiovanni, Rossella Petreschi, and Giorgio Gambosi, editors, *Algorithms and Complexity*, pages 137–149, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
- [3] Vinay A. Jawgal, V. N. Muralidhara, and P. S. Srinivasan. Online travelling salesman problem on a circle. In T.V. Gopal and Junzo Watada, editors, *Theory and Applications of Models of Computation*, pages 325–336, Cham, 2019. Springer International Publishing.

Notes/ideas for future developments:

These are some informal ideas I had regarding the development of a solution to the problem. I did not expand much on any of them aside from what is provided, and they serve as a foundation to build off of when considering the state of my findings and analysis.

- To show that breaking local 2-competitiveness does not break 2-competitiveness globally, it seems like a good idea to use potential functions. One idea is to consider the amount of "slack" we have compared to $T^G(t)$ so that we stay 2-competitive, and trying to use that to show that even if we are not 2-competitive for $I(t)$, we can offset costs and will be 2-competitive.
- I tried messing around with the potential function $\Phi(t) = 3Z^G(t) - Z^{ALG}(t)$ and some variation of it, but could not get anything to stick.
- $c(t) - d(0, e(t)) \leq C/2$, so if we can find a bound for $t - t_0$, then we show that $L(t)$ is bounded. This can be useful in showing a bound on the cost at any time t .
- We need not consider when $t > t_m$ since $Z^{ALG}(t) = Z^{ALG}(t_m)$ and $Z^G(t) = Z^G(t_m)$. This is true since otherwise, the tours on time t were longer, which can only be caused by a later request, contradicting t_m .
- Worst-case travel for the server would be to walk the entire circle to serve all requests. Given the current configuration of the algorithm, we can be pessimistic and add an additional $C/2$ to the travel to get to the first endpoint before walking all of $I(t)$ (although, since some requests will definitely be served, we can edit the algorithm to help with this). Then, if t is the release time of the latest request and $t \geq C + C/2$, we will always be 2-competitive since $t_0 + C + C/2 \leq 2t \leq 2 \times Z^{OPT}$.
- Having $T^G(t)$ be the tour that waits as long as possible is useful, but it may also be beneficial to have OL return to (or at least approach) the origin. Then, we can closer mimic $T^G(t)$ since we may potentially follow $T^G(t)$ exactly at time $Z^G(t)$ (or very close to that time).
- It is ambiguous what the server will do if the shortest path from $p(t)$ to $e(t)$ contains $I(t)$. We can have that the server walks to the other "endpoint" of $I(t)$ anyways, to have it finish at the same position on the circle as $T^G(t)$, or we can have the server just idle. In any case, this can be remedied if we have the server move to the origin before idling, whenever $I(t)$ is empty (i.e, there are no unserved requests at time t).